Week 5 Reflection

1. Draw a diagram for the following negative feedback loop: Sweating causes body temperature to decrease. High body temperature causes sweating.

A negative feedback loop means that one thing increases another while the second thing decreases the first.

Remember that we are using directed acyclic graphs where two things cannot directly cause each other.

```
Temp (1) ——(+)——> Sweat (1) ——(–)——> Temp (2) ——(+)——> Sweat (2) ——(–)——> Temp (3)
```

A directed acyclic graph can't have two things causing each other per the question. So I split up by time. For example, you're going on a run, one minute in you're hot and you start sweating. That's what cools you down. So at two minutes, you are a little bit cooler, so you start to sweat less. And then each moment causes the next one after that.

2. Describe an example of a positive feedback loop. This means that one things increases another while the second things also increases the first.

An example of a positive feedback loop are Google reviews. A restaurant with a lot of five star reviews gets prioritized when people search. Therefore, a lot of people go there. The more customers that go there means the more reviews the restaurant will get. This brings in even more customers and puts the restaurant even higher up in the search priority.

```
Reviews (week 1) ——(+)——> Customers (week 1) ——(+)——> Reviews (week 2) ——(+)——> Customers (week 2)
```

3. Draw a diagram for the following situation:

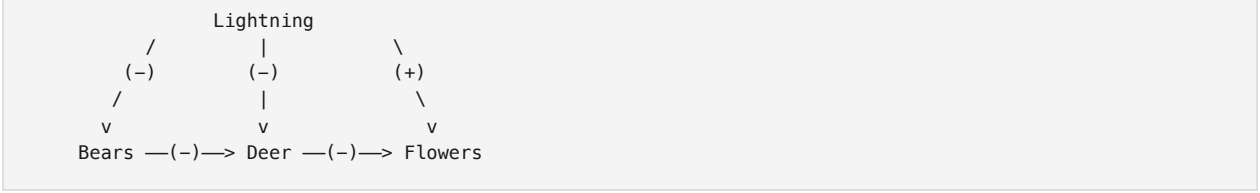
Lightning storms frighten away deer and bears, decreasing their population, and cause flowers to grow, increasing their population.

Bears eat deer, decreasing their population.

Deer eat flowers, decreasing their population.

Write a dataset that simulates this situation. (Show the code.) Include noise / randomness in all cases.

Identify a backdoor path with one or more confounders for the relationship between deer and flowers



Below is a code I used to simulate the lightning deer bear flower situation. The full code is in `week5_reflection.py`. Each row (n value) represents a season for a forest. Each deer eats 3 flowers per `3.0 * deer`.

```
rng = np.random.default_rng(0)
n = 10_000
lightning = np.maximum(rng.poisson(6, n) + rng.normal(0, 0.5, n), 0)
bears = np.maximum(20 - 1.0 * lightning + rng.normal(0, 3, n), 0)
deer = np.maximum(100 - 4.0 * lightning - 1.5 * bears + rng.normal(0, 8, n), 0)
flowers = np.maximum(600 + 25.0 * lightning - 3.0 * deer + rng.normal(0, 30, n), 0)
df = pd.DataFrame({"lightning": lightning, "bears": bears, "deer": deer, "flowers": flowers})

print(sm.OLS(df["flowers"], sm.add_constant(df[["deer"]])).fit().params["deer"])               # -6.18
print(sm.OLS(df["flowers"], sm.add_constant(df[["deer", "lightning"]])).fit().params["deer"])  # -3.04
```

The backdoor path that has one of our confounders for the relationship between deer and flowers is lightning. Deer <— Lightning —> Flowers. For example, a forest with high lightning has fewer deer and more flowers, but not due to the deer. The lightning is causing the fewer deer and more flowers.

If you are only looking at the deer and flowers, it looks like each deer eats about 6 flowers. However, once you control for the lightning, it drops to 3, which is a real number.

4. Draw a diagram for a situation of your own invention. The diagram should include at least four nodes, one confounder, and one collider. Be sure that it is acyclic (no loops). Which node would say is most like a treatment (X)? Which is most like an outcome (Y)?



```
      v               |
Buying new clothes <--┘
```

- **Treatment (X):** going to the gym.
- **Outcome (Y):** losing weight.
- **Confounder:** motivation. A person who is motivated goes to the gym more. However, a person that is motivated also eats better. Eating better alone helps them lose weight, regardless of if they go to the gym or not.
- **Collider:** buying new clothes. People who go to the gym buy workout clothes. Similarly, people who lose weight need to buy smaller clothes because their current clothes are too big.

There are two paths from the gym to losing weight:

- **Direct path:** Gym → Losing weight. This is what we actually want to measure.
- **Backdoor path:** Gym ← Motivation → Eating better → Losing weight. When we control for motivation, this path closes, leaving only the direct path, which is what we want.

We should control for motivation, but do not care about controlling for buying new clothes. The reason is, if we only looked at people who bought new clothes, we would be mixing up the gym goers and people who lost weight another way. For example, taking weight loss drugs. This would lead to the gym looking like it helps less than it actually does.

Week 6 Reflection

1. What is a potential problem with computing the Marginal Treatment Effect simply by comparing each untreated item to its counterfactual and taking the maximum difference?

A potential problem with computing the marginal treatment effect by comparing the untreated item to its counterfactual and taking the maximum difference is that the item picked is probably due to luck. Each item's Effect is a combination of its real effect with some noise added. When we take the max, we pick the item where the noise happened to be the highest, since we only are using one item. We don't average that noise out.

For example, imagine having a room full of people that flip a coin ten times. Someone will probably get nine or ten heads, but that doesn't mean their coin is better. It simply means they got lucky. With 509 untreated items, we have 509 chances for one of them to get lucky.

The quiz has no noise, so the max of 2.172 is correct. I simulated the same data with noise added to y, and the max overshot the true answer by 0.20 with little noise and 0.85 with more noise. It never came in low, meaning it's not a random error. It just happens to be too high.

2. Propose a solution that remedies this problem and write some code that implements your solution. It's very important here that you clearly explain what your solution will do.

My solution. I propose that remedies this problem is smooth max. Instead of us trusting the item with the biggest effect, I'll use all 509 treated items, and draw a line of how that effect changes with Z. From there, I will take the highest point on the line, because one lucky item will barely move from a line that's built from 509 items.

A good real-world example that this relates to would be a school that wants to know who tutoring helps most. The student whose grade jumped the highest might have had a good test day. Instead, you need to look at all the students. If students who started with lower grades improved more overall, then that is a trend we can trust more than just looking at one student.

```
_, on_untreated = unit_effects(df)
untreated = df[df["X"] == 0].assign(effect=on_untreated)

line = smf.ols("effect ~ Z", untreated).fit()
smoothed_max = line.predict(untreated).max()
```

I ran the code on 200 simulated data sets which included noise. The smooth max actually stayed within 0.005 every time whereas the plain max overshot it by 0.85. A downside to using a smoothed

max is that it draws a straight line through the data. If the real pattern is actually a curved line instead of a straight line, the answer would be off.

Week 7 Reflection

1. Create a linear regression model involving a confounder that is left out of the model. Show whether the true correlation between X and Y is overestimated, underestimated, or neither. Explain in words why this is the case for the given coefficients you have chosen.

The true correlation between X and Y is overestimated. My example is tutoring and test scores:

- **X** = hours of tutoring
- **Y** = test score
- **W** = how motivated the student is (the confounder)

Motivation affects both because students that are motivated sign up for more tutoring and they also score higher on their own. In my mock data, every hour of tutoring adds one point to the test score and motivation adds two points.

```
w = rng.normal(0, 1, N)                     # motivation
x = w + rng.normal(0, 1, N)                 # tutoring
y = 1.0 * x + 2.0 * w + rng.normal(0, 1, N)   # test score
```

Model	Effect of tutoring
True effect	1.00
Leave motivation out (`Y ~ X`)	1.94
Include motivation (`Y ~ X + W`)	1.01

If we take motivation out, then tutoring looks almost twice as good as it really is. This is because the model sees that students with tutoring scores higher, but it doesn't factor in that these students were also motivated. This means that tutoring would also take the credit of motivation. If motivation instead lowered scores, tutoring would come out at -0.08, which is underestimated, and tutoring would look like it had no effect.

2. Perform a linear regression analysis in which one of the coefficients is zero, compute the p-value of that coefficient, run the analysis 1000 times and report the best (smallest) p-value. If the p-value is less than 0.05, does

this mean the coefficient actually is nonzero? What is the problem with repeating the analysis?

In my example, W has no effect on Y.

```
w = rng.normal(0, 1, N)
x = rng.normal(0, 1, N)
y = 2 * x + rng.normal(0, 1, N)     # W is not in here
smf.ols("Y ~ X + W", df).fit().pvalues["W"]
```

I ran this analysis 1,000 times. The smallest p-value from these runs was 0.0022. 42 of the 1,000 runs were under 0.05.

No, a p-value under 0.05 does not mean the coefficient is actually nonzero. W has no effect, but in about 1 in 20 runs it passes by luck.

The problem is the more you repeat the analysis, each run has a chance to get a false positive. This means that if you run it enough times and only report the smallest fee value, there's a chance it could be the 1 in 20 runs that passes by luck.

Week 8 Reflection

1. Questions 1 and 2: Inverse probability weighting (homework_8.1.csv)

Question 1: The ATE came out to 2.27, which is closest to B (2.3).

Question 2: The first three propensity scores were [0.84, 0.58, 0.71], which is A.

```
def propensity_scores(df):
    model = LogisticRegression().fit(df[["Z"]], df["X"])
    return model.predict_proba(df[["Z"]])[:, 1]


def ipw_ate(df, p):
    treated = df["X"] == 1
    weights = np.where(treated, 1 / p, 1 / (1 - p))
    y_treated = np.average(df["Y"][treated], weights=weights[treated])
    y_untreated = np.average(df["Y"][~treated], weights=weights[~treated])
    return y_treated - y_untreated
```

A good real-world example is a job training program. People that are motivated are more likely to sign up, and they would also earn more money on their own. So if you're only comparing the people who sign up versus the ones that didn't, the job training program Gets credit for people's own motivation.

IPW fixes this problem by giving more weight to people who are rare in their group. I use logistic regression to find each person's chances of signing up based on Z. `predict_proba` gives two columns, the chance of X = 0 and the chance of X = 1. I took the second one with [:, 1]. Each treated row gets a weight of 1/P and each untreated row gets 1/(1 - P). A person that is unmotivated and signs up counts for more because there aren't many unmotivated people in the sign-up group. Finally, I took the weighted average of Y in each group and subtracted them.

2. Questions 3 and 4: Mahalanobis matching (homework_8.2.csv)

Question 3: The matched ATE came out to 3.44, which is D (3.4).

Question 4: Its nearest untreated match is at (1.52, -1.28), which is D.

```
def inverse_covariance(df):
    z = df[["Z1", "Z2"]].to_numpy().T          # 2 x N
    return np.linalg.inv(np.cov(z))            # invert the 2 x 2 covariance


for z, y in zip(treated[["Z1", "Z2"]].to_numpy(), treated["Y"]):
    d = np.array([mahalanobis(z, u, vi) for u in untreated_z])
    nearest = d.argmin()
    effects.append(y - untreated["Y"].iloc[nearest])
```

```
        distances.append(d[nearest])
        matches.append(untreated.iloc[nearest])
```

Every treated row searches every untreated row and takes the closest one. The same untreated row can be pick more then one time. This is matching with replacement.

3. Obstacles, challenges and insights

Insight: I picked Mahalanobis instead of plain distance. Let's imagine you're trying to find similar patients by weight and age. A 10-pound difference in weight isn't that big of a deal, but a 10-year age gap is. If you use plain distance, it treats them as the same. Manhalanobis scales each variable by how spread out it is.

Insight: Common support. The treated item at (2.70, 0.54) has no untreated item close to it. It's like trying to compare a 7-foot basketball player to someone that is short and didn't play. Since the closest match is so far away, that comparison is not as trustworthy as the others.

Obstacle: I got Q4 wrong the first time. I actually got question 4 wrong initially. I read "the nearest Z1 and Z2 values of the treated item" as the treated item's own values, (2.70, 0.54), and picked A because it was the closest option. The question was really asking for is the values of its nearest match, which is D.

Challenge: Getting the covariance matrix the right shape. The quiz said to put all the Z1 and Z2 values into a 2 x N matrix. My data has one row per item and one column for Z1 and Z2, which is N x 2. If you don't flip it with `.T`, `np.cov` compares all 1,000 rows to each other instead of comparing the two columns and gives a 1000 x 1000 matrix. Adding .T gives the 2 x 2 matrix that the quiz asked for.

GenAI Appendix

Overview

I will start by saying that I have been in software for over 11 years. The first half of my career I was in client-facing roles. The second half, I've been in engineering, and I'm currently a software engineering manager. Over the past 3 years, AI has completely changed how engineers code and interact with code. Most engineers now are using AI to completely write the code. You don't write the code by hand anymore. Engineers are reading the outputs from AI and ensuring the code lines up with best practice. However they are required to do a lot more now so someone who only does server engineering is also required to do some firmware and frontend engineering as an example.

I have built out a system of slash commands along with subagents. At a high level, it is Explore and Plan for the commands. I use Explore to figure out what I want to do, and then Plan will create a plan based on what Explore found. Plan will then auto-delegate to different sub-agents that I have created. For example, for a server project, I will create sub-agents like a QA agent, an API agent, a database agent, etc.

I have implemented this setup in a side software project I built with real paying customers. I have been able to integrate with 20+ year old hardware which would have taken years to do previously and with AI I was able to successfully integrate within a couple of months. It is pretty incredible what you are able to do with AI and coding now. I pushed myself and Claude a lot to accomplish this, diving deep into technical hardware concepts.

I have done a similar setup for this coding homework where I have created explore and plan commands along with different sub-agents. Since I am using Claude Code in the terminal, these sessions are not available to share like a normal chat window in a desktop UI. I have done my best to recreate the chat experiences I had with Claude Code in the ChatGPT UI windows.

Shared

(b) Claude Code Max 20x Opus Medium and ChatGPT Plus

(c) As stated in the overview, I use AI as a coding assistant to write the code for me, which I then review. I have a workflow built out where I explore with the agent, come up with a plan, and then implement and reiterate from there.

(d) As stated in the overview, I use AI to not only save time but also push myself and the code in ways that I wouldn't be able to do on my own.

Week 5

(a) https://chatgpt.com/share/6ab7c9d7-7c3c-83e8-a215-e14496c88141

Week 6

(a) https://chatgpt.com/share/6ac1331d-87b0-83e8-b64e-e0adf25dc2b9

Week 7

(a) https://chatgpt.com/share/6accd8dc-9a74-83e8-9f3f-b62e5b6ba1bc

Week 8

(a) https://chatgpt.com/share/6adb7247-7244-83e8-bb1a-c940787b9794